

my-first-harness-project

research

- DSH
 - Cordis
- Headlong agent
- context anchoring

Ideas

- 프로젝트를 자기 정체성으로 가지는?
- 남은 토큰 사용량을 바탕으로 루프를 자체 수정하는?
- 컨텍스트를 latent space로 임베딩해 기본 단어와의 유사도로 수치를 산출해서 컨텍스트에 넣는
- 아니면 게임 하네스를 만들어버려. 하네스 레벨에서부터 게이미피케이션을 추가한 버전으로. 토큰 가능하게.
 - 게임 토큰을 활성화하고 시작하면 하네스 자체에서 게임 형식으로 자기한테 맞는 하네스 구조를 찾아가는거지. 하bti처럼. 그렇게 찾고나면 다시 토큰 끄고 코딩할 때 쓰면 되고. 그걸 프로파일로 저장해서 왔다갔다 할 수도 있고.

DSH를 공부해서 구현하려고 했는데

이해가 안된다!

그래 대체 DSH이해도 안되고 용어도 헛갈리고 그래서 일단 만들기 시작하면서 하나하나씩 배우는 방식으로 바꿀거야. 네가 해줘야 할 것은

진짜 겉가지 다 빼고 핵심중의 핵심, 코어중의 코어만 남기고 하네스라는걸 이해하게 프로젝트를 진행해주는거야.

언어는 타입스크립트를 쓸거고, Cordis를 써서 DSH의 철학을 유지하기는 할거야. 다만 개발 방식을, 시작부터 모든걸 고려하고 확장 가능한 DSH급의 코드를 하나씩 쓰는게 아니고 '최소 구현 -> 이해 -> 실제 사용 -> 문제점 발견 -> DSH등 실제 상용 하네스의 솔루션 필요성 이해 -> 최소구현' 이런식의 개념 학습 이해를 목적으로한 실습 루프로 돌거야. 어디서 시작할까?

--> 완전 기초, 빈 파일부터 시작해서 하나씩 쌓아보자. 에러처리나 확장성 고려 하나도 안하고 진짜 최소한의 코드로 구현하며 학습해보자.

Implementation logs

1. 실행 시 인자를 유저 입력으로 ai proxy 사내 api 활용해 fetch함수로 http호출해서 LLM 응답 받기

```
choechiwon@chiwonchoi-macos my-first-harness % node my-first-harness.ts
> 카운터 3번 올려줘

[
  {
    role: 'system',
    content: '너는 마스터를 돕는 비서다. 마스터의 요구를 만족하라. \n' +
      '          - 사용할 수 있는 도구: getCurrentTime, getOtherLLMsOpinion, counterUP, getCounterVal\n' +
      '          - 도구 사용법: 어떠한 수정이나 다른 출력도 없이 오직 'CALL {도구이름}'으로 답하라. 예를 들면, 'CALL getCurrentTime''
  },
  { role: 'user', content: '카운터 3번 올려줘' }
]

[
  {
    role: 'system',
    content: '너는 마스터를 돕는 비서다. 마스터의 요구를 만족하라. \n' +
      '          - 사용할 수 있는 도구: getCurrentTime, getOtherLLMsOpinion, counterUP, getCounterVal\n' +
      '          - 도구 사용법: 어떠한 수정이나 다른 출력도 없이 오직 'CALL {도구이름}'으로 답하라. 예를 들면, 'CALL getCurrentTime''
  },
  { role: 'user', content: '카운터 3번 올려줘' },
  { role: 'assistant', content: 'CALL counterUP' },
  {
    role: 'user',
    content: 'TOOL RESULT : successfully increased counter'
  }
]
```

```

    }
  ]
  [
    {
      role: 'system',
      content: '너는 마스터를 돕는 비서다. 마스터의 요구를 만족하라. \n' +
        '      - 사용할 수 있는 도구: getCurrentTime, getOtherLLMsOpinion,
counterUP, getCounterVal\n' +
        "      - 도구 사용법: 어떠한 수정이나 다른 출력도 없이 오직 'CALL {도구이름}'으로
        답하라. 예를 들면, 'CALL getCurrentTime'"
    },
    { role: 'user', content: '카운터 3번 올려줘' },
    { role: 'assistant', content: 'CALL counterUP' },
    {
      role: 'user',
      content: 'TOOL RESULT : successfully increased counter'
    },
    { role: 'assistant', content: 'CALL counterUP' },
    {
      role: 'user',
      content: 'TOOL RESULT : successfully increased counter'
    }
  ]
  [
    {
      role: 'system',
      content: '너는 마스터를 돕는 비서다. 마스터의 요구를 만족하라. \n' +
        '      - 사용할 수 있는 도구: getCurrentTime, getOtherLLMsOpinion,
counterUP, getCounterVal\n' +
        "      - 도구 사용법: 어떠한 수정이나 다른 출력도 없이 오직 'CALL {도구이름}'으로
        답하라. 예를 들면, 'CALL getCurrentTime'"
    },
    { role: 'user', content: '카운터 3번 올려줘' },
    { role: 'assistant', content: 'CALL counterUP' },
    {
      role: 'user',
      content: 'TOOL RESULT : successfully increased counter'
    },
    { role: 'assistant', content: 'CALL counterUP' },
    {
      role: 'user',
      content: 'TOOL RESULT : successfully increased counter'
    },
    { role: 'assistant', content: 'CALL counterUP' },
    {
      role: 'user',

```

```

    content: 'TOOL RESULT : successfully increased counter'
  }
]
[
  {
    role: 'system',
    content: '너는 마스터를 돕는 비서다. 마스터의 요구를 만족하라. \n' +
      '    - 사용할 수 있는 도구: getCurrentTime, getOtherLLMsOpinion, counterUP, getCounterVal\n' +
      "    - 도구 사용법: 어떠한 수정이나 다른 출력도 없이 오직 'CALL {도구이름}'으로\n"
      '    답하라. 예를 들면, 'CALL getCurrentTime''
  },
  { role: 'user', content: '카운터 3번 올려줘' },
  { role: 'assistant', content: 'CALL counterUP' },
  {
    role: 'user',
    content: 'TOOL RESULT : successfully increased counter'
  },
  { role: 'assistant', content: 'CALL counterUP' },
  {
    role: 'user',
    content: 'TOOL RESULT : successfully increased counter'
  },
  { role: 'assistant', content: 'CALL counterUP' },
  {
    role: 'user',
    content: 'TOOL RESULT : successfully increased counter'
  },
  { role: 'assistant', content: 'CALL getCounterVal' },
  { role: 'user', content: 'TOOL RESULT : 3' }
]

```

카운터가 세 번 증가하여 현재 값은 3입니다.

현재까지 된 것:

- 사용자 목표
- LLM 판단
- 툴 선택
- 하네스가 툴 실행
- 결과를 LLM에 전달
- LLM이 다음 행동 선택
- 목표 확인
- 스스로 최종 답변
- 종료

느낀점:

- 핵심 루프는 정말 별거 없구나. 프롬프트에 전부 의존하는 구조라서.
- LLM출력을 구분할 필요가 있다. 툴 호출 요청이 그냥 답변칸에 나오니까 툴에 인자를 넣으려면 툴별로 파서를 만들거나 해야돼서 귀찮다.
- 이쯤 해보니 드는 잡생각:
 - 나는 잘 만들어진 codex랑 작업해서 귀찮은걸 다 맡기고 편한데, 실제 개발회사에서는 자기들이 만드는게 최신이니까, 빨리 잘 만들어서 그걸로 다시 하네스 개발하는 루프를 서서히 돌리는 걸 하려고, 하고 있었겠다.
 - 근데 이게 마치 컴파일러나 새 언어 개발할 때 부트스트랩 방식으로 하는거랑 같은 구조겠다~

노트

- `response.json().content` -> 여기서 content가 정해진 HTTP응답 규약이 아니고 그냥 AI proxy에서 정해둔 필드 이름이었다. body안에 구조나 파일형식은 뭐든 가능하고 이건 API provider가 정하는 것. 그래서 openAI API가 좀더 구조가 복잡하고, 앞서 내가 느꼈던 문제점들을 효율적으로 해결 가능하다.
 - 다음 단계는 이걸로 고쳐봐야겠다.
 - 가장 헷갈렸던게 왜 api 호출할 때 tool 정의, 응답받을 때 여러 필드들을 그냥 쓸 수 있는거지? 였는데 사실 기본은 그거 없이 content로만 파싱해서 쓰는거고, API provider에서 그걸 편하게 하도록 내부적으로 알아서 처리하는 거였다. 따라서, 원래 하네스를 만들 때는 DSH처럼 공통 규격을 만들어 provider별로 어댑터를 만들던가, 아님 provider에 맞게 공식문서를 읽어서 그 구조대로 하네스 코드 자체를 설계하던가 고치던가 해야된다.

5. 실제 openAI API호출 규격으로 툴 입력, 호출 리팩토링.

- 실제 응답은 이렇게 생겼었다.
- API 호출 목표에 따라 provider쪽에서 제공하는 endpoint가 다르고, 그에따라 내부 동작, 답변 필드 구조도 바뀐다. 예를들어 아래 답변은 /chat/completion 답변이라 sessionID같은게 없음.
 - 하네스 자체 세션 개념과 API provider의 session 개념이 다르다.

```
{
  id: "chatcmpl-EKDNIXqNG6RiJQiI3ZqJES9W0zTIW",
  object: "chat.completion",
  created: 1788486680,
  model: "gpt-4o-2024-08-06",
  choices: [
    {
      index: 0,
      message: {
        role: "assistant",
        content: "Hello! How can I assist you today?",
      },
    },
  ],
}
```

```

        refusal: null,
        annotations: [],
    },
    logprobs: null,
    finish_reason: "stop",
},
],
usage: {
    prompt_tokens: 138,
    completion_tokens: 10,
    total_tokens: 148,
    prompt_tokens_details: { cached_tokens: 0, audio_tokens: 0 },
    completion_tokens_details: {
        reasoning_tokens: 0,
        audio_tokens: 0,
        accepted_prediction_tokens: 0,
        rejected_prediction_tokens: 0,
    },
},
service_tier: "default",
system_fingerprint: "fp_e3057867b6",
};

```

- 이제 툴 호출 시 인자까지 LLM이 생성해서 넘겨줄 수 있다.
 - 근데 여기까지 쓴 코드가 싹다 합쳐도 200줄이 안된다. 아직까지도 하네스의 본질은 그냥 우리가 필요하거 만들어서 LLM API 요청 메시지에 dump한 다음 알아서 써달라고 하는 거에서 벗어나질 않네. 사실상 API provider가 다 만들어준거 갖다쓰는거에 불과.
6. ToolManager 구현. 현재 툴을 추가하려면 1. 실제 구현 2. 시스템 프롬프트에 추가 3. 사용시 switch case문에 추가 로 번거롭다. 추가로 툴 실행에도 for loop 추가해서 여러 툴 호출 대응 구현.
 - 이제 새로운 tool을 구현하면, ToolManager 인스턴스에 register만 하면 된다.
 7. Context 조립 구현. 지금 API 호출 부분에서 직접 넣던걸 assembleContext 함수로 따로 뺌. 앞으로 여기서 확장할 수 있게. 예를들면, 이제부터 messages는 영구적인 기록이고, 이번 턴만 넣는건 runtimeContext로 추가해서 호출함.
 - 이제 스킬 시스템 만들 수 있다.
 8. skill 시스템 구현. skillManager에 등록하면 요청시 조립돼 들어감.
 - 툴 사용과 동시에 응답하는 content도 출력하도록 해서 턴 중간 보고 가능하게
 - 했는데 애가 자기 하네스 구조를 몰라서 답답하게 작동하고 시스템프롬프트에 설명 써줘야 함. 이제 그냥 파일 읽을 수 있는 툴을 추가하자.
 9. readTextFile 툴 추가. 이제 자기 구현 파일을 읽을 수 있음. 그걸 컨텍스트에 넣고 명령하니까 좀 쉽게 된다.
 10. /quit 유저 커맨드 추가.

11. 핵심 툴들 추가. 경로의 파일 읽기, 파일 쓰기, 터미널 실행 툴까지.

이제 이쯤 오니까 대충 다 된것 같다. 스킴, 툴, mcp연결, 에러처리, 가드레일, 세션 등등 더 구현할 수 있는데 핵심은 지금부터는 하네스라는 개념을 구성하는 코어에서 벗어나 설계 철학 선택에 해당한다는 것.

가장 핵심적인 하네스에는 플러그인이 없어도 된다. 필수 책임은 이것뿐이다.

사용자 입력을 받는다

- 모델에게 컨텍스트를 보낸다
- 모델 응답을 받는다
- 툴 요청이면 실행하고 결과를 다시 넣는다
- 완료할 때까지 반복한다
- 최종 답변을 출력한다

이건 네 코드가 이미 구현했다.

각 책임을 어떻게 구현할지부터 설계 철학이 생긴다.

모델 호출이 필요하다

- 한 provider에 직접 맞춤 것인가
- 공통 인터페이스와 provider adapter를 둘 것인가

대화 기록이 필요하다

- messages 배열만 둘 것인가
- append-only session event log를 둘 것인가

툴 실행이 필요하다

- switch로 실행할 것인가
- ToolManager registry를 둘 것인가
- MCP까지 지원할 것인가

컨텍스트가 필요하다

- 모든 정보를 항상 넣을 것인가
- 필요한 스킴만 gradual disclosure할 것인가
- 오래된 내용을 요약할 것인가

기능 확장이 필요하다

- 메인 파일을 직접 수정할 것인가
- 외부 플러그인만 추가할 것인가

그럼 이제 플러그인 구조를 만들고, 파일을 깔끔하게 분리한 다음, 세션 만들고, mcp 연결하고, Cordis 도입하고 이정도면 되지 않을까?

12. 우선 모듈화부터 출발. 플러그인의 탄생 동기가 여기에 있다고 해서. 모듈화를 하고 나니 모듈을 그냥 다운받아 쓸 수 없다. 왜냐하면 메인에서 다운받은 툴을 toolManager에 register할 때 이름,

description, param 등등을 직접 써야됨. 이걸 toolManager를 인자로 써서 직접 등록하는 함수를 모듈에 추가해 해결. 이제 모듈을 그냥 다운받아 직접 등록하는 함수만 import해서 쓰면 툴들을 쓸 수 있다.

- 스킬도 모듈에 추가, 스킬 폴더에서도 로드 가능하게. 이제 모듈은 스킬/툴 단위가 아니고 클로드 코드 플러그인 같은 기능 단위고, 스킬 매니저, 툴 매니저가 register하는걸 공통 등록포인트로 여러 경로에서 등록 가능하다.

13. 다음으로는 세션을 만들어보자. 세션 ID로 구분되는 메모리(메시지 에레이)를 세션으로 감싸서. 이제 /new로 대화를 초기화 가능. 런타임 꺼져도 기록되고, id로 resume가능.

이제 온보딩 과제는 완료.

온보딩 프로젝트 - 자체 하네스 제작

- 시스템 프롬프트 기능, 세션 기능, 툴 기능, 스킬 기능 정도
 - API가 돌아갈때 들어가야되는 툴과 컨텍스트,
- 위 목표 습득 지식을 습득하는게 목적

다음 과제

1. harness 보완하기

1. 우선 하네스 미구현된 부분부터 보완
2. 컨텍스트 컴팩션
 1. 겁나 큰 용량의 이미지가 들어갔다고 치면 그거 계속 살려둘 것인가.
3. 툴콜 보호
4. 캐싱
5. 툴 스트리핑
6. 아마 베이커리 코덱스 팜을 쓰면 chat completion은 못쓸거다.
7. luna 뿐만 아니라 클로드 haiku를 쓰는 버전도 전환할 수 있게 만들어봐라.
 1. 클로드의 컨텍스트를 쓰는 방식이 더 정석적이다.
 2. 둘이 API 호출이나 컨텍스트를 쓰는 방식이 달라서 차이를 공부해볼 가치가 있을 것

2. 원샷으로 테트리스 만들어 증명하기

3. 하네스가 직접 테트리스 플레이 가능하게 만들기

그럼 우선 하네스 보완부터 이어서 해보자.

산출물 계획

- 자체 테스트, 보완 루프 없이 원샷 생성 테트리스
 - 외부 skill, tool, mcp 사용한 버전, 안한 버전
- 자체 테스트, 보완 루프를 돌린 테트리스
 - 외부 skill, tool, mcp 사용한 버전, 안한 버전

- 이걸 haiku, luna로
- 총 8개 버전 생성 가능

Implementation Logs - 2

1. 작업폴더, 사용자 전역 폴더(세션 이제 여기에 저장), 프로젝트 폴더 분리 구현
 1. 경로 문제로 bun 빌드 실행 안되고 있었음
 2. 이제 skill 두 군데/권한 에서 로드 가능
2. history(원문 보존), messages(API에 보내는 압축될 수 있는 메시지) 분리. 컨텍스트 요약/기준치 이상에서 자동 호출 구현.
 1. 컨텍스트 요약도 LLM을 활용하므로 llm.ts로 모듈 분리. context-manager.ts추가.
 2. 단순 LLM 요약 요청 뿐만 아니라, 툴 호출 결과가 기준치 이상인 것들 앞뒤만 남기고 자르는 DSH 방식 도입.
3. 툴 호출 가드 구현
 1. 모델이 없는 툴 이름을 호출하거나, 인자를 잘못쓰거나, json 파싱이 안되면 실행 전 검증해 에러 메시지를 툴 결과 대신 기록한다.
 2. 요청 검증은 통과했는데 실행 결과가 에러인 경우도 터지는게 아니라 에러 메시지 반환하도록 수정.
4. multi API provider 지원 구현
 1. 하네스 내부 공통 형식 정의, 기존 chat-completion api부터 adapter 구현(+ AGENTS.md 추가)
 2. openAI responses api + 5.6 luna adapter 구현. ai proxy 사용.
 3. anthropic messages api + haiku adapter implemented. used ai proxy
 4. 모델 선택 분리: model-config.ts
 1. 기본 실행은 Luna, node my-first-harness.ts haiku는 Haiku.
5. mcp 연결 구현
 1. @modelcontextprotocol/sdk sdk 패키지 설치해서 활용. 통신 방식에는 stdio(local server와 통신. 자체 구현뿐만 아니라 로컬에 코드 다운받아서 mcp로 연결해 쓸 수 있게 배포하기도 함. 포트번호 관리같은거 필요없어서 좋음), http 두 가지가 있음
 2. sdk로 client, server(transport)의 connection 쌍을 서버당 하나씩 만들. tools 로드, tool 호출, 마지막에 정리에 사용
 3. 핵심은 mcp라고 별게 아니고, 서버한테서 'mcp' 라는 규격에 맞게 툴 리스트가 제공되는걸 받아와서, 기존 내장 툴들 처럼 ToolManager에 등록해서(name, description, parameters, execute) 쓰는건데, 이때, execute함수가 내장 함수 호출이 아니고, client.callTool로 연결된 서버에 호출하는 것임.
6. 실제로 인터넷에 공유되는 skill들을 다운받아 쓸 수 있게 만들면서 progressive disclosure도 구현하기
 1. yaml frontmatter 파싱, 기존 name, instruction 전문에서 name, descriptions, location 으로 바꿔서 필요시 readfiles tool로 로드하도록.

2. tool 사용 시 로그 출력하도록
7. history expanding. From previous json to jsonl + other logs(including tool calling, usage, etc which were excluded so far)
8. 툴 백그라운드 실행 옵션 구현. 이제 모델이 필요에 따라 툴을 백그라운드로 실행하고 툴을 이어갈 수 있음. 실행한 툴은 id를 받아 출력과 상태를 확인하고, kill할 수 있음.
9. bakery farm adapter, set as a default
10. edit text file tool 추가. 기존 무조건 파일 전체 덮어쓰우기에서 글자단위 수정 가능하게.
11. 이미지 입력 구현
 1. /attach {파일경로} 로 사용자가 cli에서 첨부 가능. 이러면 다음 사용자 메시지 전송에 이미지 파일 같이 감
 2. 내부적으로는 loadImage함수가 이미지를 base64같은 api가 지원하는 형식 데이터로 바꿔서 이미지 경로와 함께 api호출 시 첨부. 멀티모달 지원 api, model에서만 작동
12. 리팩토링!!
 1. 지금 my_first_harness.ts의 메인 코드가 너무 의존성이 심하고(메인 루프가 우리 코드들을 쓰는데 전부 import를 가져와서 그대로 쓰고 추상화 계층이 하나도 없어서) 코드가 크다. llm adapter, toolManager, skillManager, history, paths를 파라미터로 생성하는 agent 객체를 정의해서 agent.ts에 코어로 두자. 이렇게 하면 코어는 따로 빠지고 사용자 인터페이스는 cli, tui, web 만들어서 코어 연결해 쓰는 식으로.
 2. 콘솔 로깅도 메인 루프에서 직접 찍는게 아니라, 이벤트 입출력으로 추상화 해서 인터페이스 레벨에서 이벤트 받으면 로그 찍는 식으로.
 3. 테스트 겸 추가로 tui구현함. 앞으로 tui쓸 것.
13. 다음으로 브라우저를 쓸 수 있었으면 좋겠어서 playwright mcp 다운받아 등록
 1. 이때, 현재 mcp client에서는 mcp응답 이미지 무시하고 있었는데 기존 형식으로 받을 수 있게 업데이트.
14. tools, skills, plugins, mcps 세션 중간에 토글할 수 있는 tui커멘드 추가
15. 컨텍스트 관리 보완
 1. 지금은 모델에 무관하게 하드코딩한 기준으로 compaction을 트리거함. 요약도 메시지 배열 전체를 넘기는 중. 모델별 컨텍스트 용량을 반영하고, 요약은 최신 기록(컨텍스트 예산 전체의 16%까지)은 유지하고 오래된 것만 잘라서 요약하도록 수정.
 2. AGENTS.md 지원, claude code처럼 ToolSearch tool만들어서 기존처럼 모든 툴 한번에 넣는게 아니고 mcp는 이름만 넣고 필요시 모델이 점진적 공개 하도록 수정.
16. 히스토리 이미지 관리 개선
 1. 지금은 8메가 넘으면 그냥 더이상 이미지 첨부 안되게 돼있음
 2. 이미지 원본 보관, 해상도 너무 클 시 압축, 20MiB넘으면 오래된 것부터 요약 메시지로 대체 구현
17. plan, edit모드 구현
 1. 이걸 위해선 우선 권한부터 구현해야 한다. 지금은 권한기능, 관리 없음

2. DSH에서는 모드랑 권한이 나뉘져 있어서 `session.mode`로 모드 인스트럭션 프롬프트를 시스템 컨텍스트 마지막에 추가하고, `session.permission` 으로 톨 요청마다 권한 게이트 통과하는식으로
3. plan모드용 `exit_plan_mode` 톨 추가
4. 써보니까 매번 일회성 승인이라 불편해서 세션 단위 승인 옵션, yolo모드 추가

18. TUI 개선

1. 스크롤이 안됨, 턴 중간에 `esc`로 중단이 안됨(이건 에이전트 코어에 `signal` 추가해야 하는 큰 수정, `interrupt`로 작동하도록), 톨 실행 결과 표시

19. 시스템 프롬프트들 상용 하네스 버전 참고해서 최적화

1. 클로드 코드 시스템 프롬프트같은거 참고해서 고치고, 테스트하고, 비교해서 최적화하는 루프 요청
2. 을 시켰는데 특별한 개선은 없었다. luna는 원래 잘했고 haiku는 원래 못했는데 바뀌도 유지. 모델 성능이 제일 중요한 듯

20. 게임 테스트 플러그인 구현

1. 하네스가 게임을 제작하고 직접 사람처럼 플레이까지 하며 테스트를 할 수 있어야 한다
2. 테스트를 구현하는 방법에는 크게 4가지가 있는데
 1. 게임 코드를 이해하고 자동화 코드 작성하는 해킹(테트리스 테스트 그냥 시키니까 이렇게 해서 무한히 점수 얻음)
 2. playwrightMCP나 computer use같은걸로 화면 캡처 -> 모델 응답 -> 실행 루프(실제 인간과 가장 유사하나 테스트 해보니 너무 느려서 플레이 안됨)
 3. 모델이 플레이하는 코드를 작성하고, 코드 자체를 수정하는 루프로 플레이(라는 것도 있더라)
 4. 게임 플레이만을 위해 학습된 고속 모델 사용(도 연구중이라던데 아직 바로 가져다 쓸만 한게 없어 보인다더라, codex 피셜)
3. 그래서 2번 방법을 쓰되, playwright의 `page.clock` 기능으로 모델이 플레이 할 수 있는 수준으로 게임의 진행속도를 조절한다.
 1. 이런 방법론과 필요한 코드를 스킴, 톨로 만들어 하나의 플러그인으로 설치해두고 테스트 해보자

끝!!!

남은 것들

- 샌드박스 실행
- 스트리밍 응답 지원
- 세션 fork
- 플러그인, mcp 인터넷 마켓플레이스 연결

- web search 같은 기본적인 툴, 스킬 등 보완